

Willkommen zum Modul Grundlagen Verteilter Systeme

Michael Schöttner

SS 2024



1. Einführung

Michael Schöttner

Grundlagen Verteilter Systeme

1.1 Organisation

- Vorlesung ab 11.4.2024
 - Donnerstags, 10:30 – 12:00 Uhr in 5G
- Vorlesung/Übung ab 12.4.2022
 - Freitags, 8:30 – 10:00 Uhr in 5G
 - Übungsaufgaben sind freiwillig
- Klausurtermine:
 - 1. Klausur: 23.7.2024
 - 2. Klausur: 17.9.2024



1.1 Organisation

- Kommunikation über ILIAS und RocketChat
- Materialien in ILIAS
- Modul umfasst 5 ECTS, für den Bachelorstudiengang
- Personen
 - Vorlesung: Prof. Dr. Michael Schöttner
 - Übungen: Msc. Fabian Ruhland



1.2 Einordnung des Moduls

Bachelor

Betriebssysteme und Systemprogrammierung, 4V+2Ü, 10 LP

Grundlagen Verteilter Systeme, 2V+2Ü, 5 LP

Master

Betriebssystem-Entwicklung,
1V+3Ü, 5 CP

Aktuelle Trends in der Systemsoftware,
2S, 5 CP

Isolation und Schutz in Betriebssystemen,
1V+3Ü, 5 CP



1.3 Lernziele

- Tieferes Verständnis für die grundlegenden Herausforderungen und Lösungen beim Bau verteilter Systeme
- Vermittelte Konzepte in eigenen Worten erklären können
- Erlernte Konzepte in theoretischen und praktischen Übungen anwenden können
- Eine genauere Auflistung findet sich in der Modulbeschreibung.



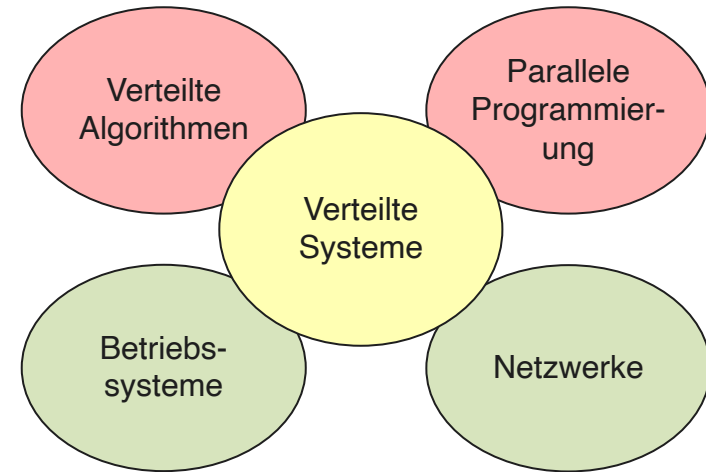
1.4 Motivation

- Ausgangssituation
 - Es gibt überall immer mehr Computer die Daten erzeugen
 - Immer mehr verteilte Daten müssen schnell verarbeitet werden
 - Viele Anwendungen müssen weltweit (hoch)verfügbar sein und manche haben Millionen oder gar Milliarden an interaktiven Benutzer*innen
- Auch ein teurer Super-Computer reicht hierfür oft nicht, daher sind verteilte Systeme unvermeidlich



„Verteilte Systeme“ ist ein Querschnittsthema

- Hieraus ergibt sich ein Nutzen in vielen Bereichen
- (Kleinere Überschneidungen mit anderen Vorlesungen unvermeidlich)



1.5 Definition: „Verteiltes System“

- “A distributed system is one in which the failure of a computer you didn’t even know existed can render your own computer unusable.”, L. Lamport
- “A distributed system is a collection of independent computers that appears to its users as a single coherent system”, A. Tanenbaum
- „Autonome Rechner bilden ein verteiltes System, wenn Sie durch ein Netzwerk verbunden sind und gemeinsam eine Aufgabe lösen“



1.6 Ziele

- **Einfacher Zugriff auf entfernte Ressourcen**
 - WebServices, Daten, Virtuelle Maschinen, ...
- **Leistungsgewinn**
 - mehr Ressourcen (CPU, Speicher, Disk), sofern Anwendungen dies nutzen können
- **Zuverlässigkeit**
 - Redundanz durch viele Rechner
- **Flexibilität**
 - inkrementelle Erweiterbarkeit, sofern System / Anwendungen skalieren
- **Kosten**
 - Aggregation vorhandener Ressourcen ist billiger als ein teurer Super-Computer



1.7 Herausforderungen

- **Komplexität**
 - Verteilte-Software ist erheblich komplexer
 - Nebenläufigkeit, Nicht-Determinismen
 - Parallelität auf jedem Knoten
- **Heterogenität**
 - Hardware
 - Betriebssysteme
 - Bibliotheken, ...
- **Skalierbarkeit** bezüglich Serveranzahl ist gewünscht
 - Aber erzielbare Leistung wächst nicht beliebig mit der Anzahl an Servern!



1.7 Herausforderungen

- **Fehlertoleranz:**
 - Die Fehler-Wahrscheinlichkeit steigt mit der Anzahl der Server!
 - In großen verteilten Systemen sind Fehler keine Ausnahme, sondern die Regel
- **Netzwerk:** stark abhängig vom Anwendungsfall
 - langsam bis schnell; aber immer langsamer als ein lokaler Datenzugriff
 - Nachrichten können verloren gehen
 - Netzwerke können partitioniert sein
- **Sicherheit:** besondere Maßnahmen nötig
 - Verschlüsselung, Authentisierung, ...



1.8 Inhalt

- Architekturen
 - Cluster, Cloud, Edge, Fog, ...
- Grundlagen
 - Prozesse & Threads
 - Sperren
- Kommunikation
 - Sockets, Multithreading, Server-Skalierbarkeit
 - Entfernter Prozeduraufruf (gRPC)
 - Distributed Shared Memory



1.8 Inhalt

- Zeit
 - Uhrensynchronisierung
 - Logische Zeit
- Globale Zustände
 - Chandy Lamport Snapshot
 - Deadlock-Erkennung
- Replikation und Konsistenz
 - Strenge und schwache Konsistenz
 - Replikatverwaltung und Aktualisierungsmethoden



1.8 Inhalt

- Fehlertoleranz
 - Fehlermodelle
 - CAP Theorem
 - Checkpointing & Recovery vs. Recompute
- Schwache Konsistenz und Skalierbarkeit
 - Peer-to-Peer: Gnutella, Chord, Pastry
 - Key-Value-Stores: Amazon Dynamo, DXRAM
 - Dateisysteme: Google File System



1.8 Inhalt

- Strenge Konsistenz und Skalierbarkeit
 - (Verteilte) Transaktionen
 - FLP Theorem
 - Paxos
 - BFT
- Sicherheit
 - Verschlüsselung
 - Digitale Signatur



1.8 Inhalt

- Fallstudien
 - MapReduce
 - In-Memory Computing
 - Stream Processing



1.9 Literatur

- George Coulouris et.al.: „Distributed Systems, Concepts & Design”, Pearson, 5. Aufl., 2011.
- M. van Steen & A. Tanenbaum: „Distributed Systems”, CreateSpace Independent Publishing Platform; Auflage: 3.01 (1. Februar 2017)

